

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn import metrics
import seaborn as sns
```

```
import kagglehub
# Download latest version
path = kagglehub.dataset_download("salihacur/diabetes")
print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'diabetes' dataset.  
Path to dataset files: /kaggle/input/diabetes

```
import os
print(os.listdir(path))
```

```
['diabetes.csv']
```

```
df = pd.read_csv(f"{path}/diabetes.csv")
print(df.shape)
df.head()
```

```
(768, 9)
```

|   | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | BMI  | DiabetesPedigreeFunction | Age | Outcome |
|---|-------------|---------|---------------|---------------|---------|------|--------------------------|-----|---------|
| 0 | 6           | 148     | 72            | 35            | 0       | 33.6 | 0.627                    | 50  | 1       |
| 1 | 1           | 85      | 66            | 29            | 0       | 26.6 | 0.351                    | 31  | 0       |
| 2 | 8           | 183     | 64            | 0             | 0       | 23.3 | 0.672                    | 32  | 1       |
| 3 | 1           | 89      | 66            | 23            | 94      | 28.1 | 0.167                    | 21  | 0       |
| 4 | 0           | 137     | 40            | 35            | 168     | 43.1 | 2.288                    | 33  | 1       |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.isnull().sum()
```

|                                 | 0 |
|---------------------------------|---|
| <b>Pregnancies</b>              | 0 |
| <b>Glucose</b>                  | 0 |
| <b>BloodPressure</b>            | 0 |
| <b>SkinThickness</b>            | 0 |
| <b>Insulin</b>                  | 0 |
| <b>BMI</b>                      | 0 |
| <b>DiabetesPedigreeFunction</b> | 0 |
| <b>Age</b>                      | 0 |
| <b>Outcome</b>                  | 0 |

```
dtype: int64
```

```
feature_cols = ["Pregnancies", "Glucose", "BloodPressure", "Insulin", "BMI", "Age",
                "DiabetesPedigreeFunction"]
x = df[feature_cols]
y = df.Outcome
print(x)
print(y)
```

|     | Pregnancies | Glucose | BloodPressure | Insulin | BMI  | Age | \ |
|-----|-------------|---------|---------------|---------|------|-----|---|
| 0   | 6           | 148     | 72            | 0       | 33.6 | 50  |   |
| 1   | 1           | 85      | 66            | 0       | 26.6 | 31  |   |
| 2   | 8           | 183     | 64            | 0       | 23.3 | 32  |   |
| 3   | 1           | 89      | 66            | 94      | 28.1 | 21  |   |
| 4   | 0           | 137     | 40            | 168     | 43.1 | 33  |   |
| ..  | ...         | ...     | ...           | ...     | ...  | ... |   |
| 763 | 10          | 101     | 76            | 180     | 32.9 | 63  |   |
| 764 | 2           | 122     | 70            | 0       | 36.8 | 27  |   |
| 765 | 5           | 121     | 72            | 112     | 26.2 | 30  |   |
| 766 | 1           | 126     | 60            | 0       | 30.1 | 47  |   |
| 767 | 1           | 93      | 70            | 0       | 30.4 | 23  |   |

```

DiabetesPedigreeFunction
0      0.627
1      0.351
2      0.672
3      0.167
4      2.288
..      ...
763     0.171
764     0.340
765     0.245
766     0.349
767     0.315

[768 rows x 7 columns]
0      1
1      0
2      1
3      0
4      1
..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64

```

```

x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_state=0)
print(x_train.shape,y_train.shape,x_test.shape,y_test.shape)

```

```

(576, 7) (576,) (192, 7) (192,)

```

```

model = LogisticRegression(solver='lbfgs',max_iter=1000)
model.fit(x_train,y_train)

```

```

▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)

```

```

y_pred = model.predict(x_test)
print(y_pred)

```

```

[1 0 0 1 0 0 1 1 0 0 1 1 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0
 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 1 1 1 1 0 0 0 0 0 0 1
 1 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1 0
 0 1 0 1 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0
 0 0 0 1 0 0 1 0 1 0 0 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0
 0 1 0 0 0 0 0]

```

```

conf_mat = metrics.confusion_matrix(y_test,y_pred)
print(conf_mat)
accuracy = metrics.accuracy_score(y_test,y_pred)
print(accuracy)

```

```

[[118  12]
 [ 26  36]]
0.8020833333333334

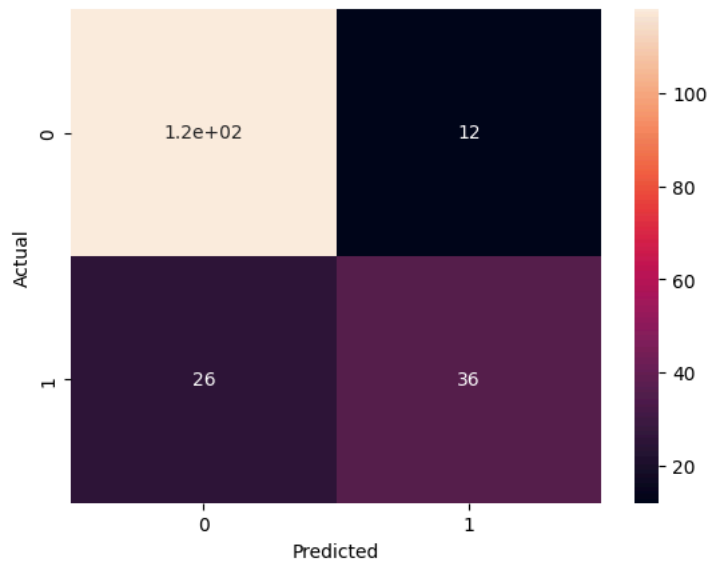
```

```

conf_mat = pd.crosstab(y_test,y_pred,rownames=["Actual"],colnames=["Predicted"])
sns.heatmap(conf_mat,annot=True)

```

<Axes: xlabel='Predicted', ylabel='Actual'>



```
url = "https://stats.idre.ucla.edu/stat/stata/dae/binary.dta"
df = pd.read_stata(url)
df.head()
```

|   | admit | gre   | gpa  | rank |  |
|---|-------|-------|------|------|--|
| 0 | 0.0   | 380.0 | 3.61 | 3.0  |  |
| 1 | 1.0   | 660.0 | 3.67 | 3.0  |  |
| 2 | 1.0   | 800.0 | 4.00 | 1.0  |  |
| 3 | 1.0   | 640.0 | 3.19 | 4.0  |  |
| 4 | 0.0   | 520.0 | 2.93 | 4.0  |  |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.isnull().sum()
```

```

0
admit 0
gre    0
gpa    0
rank   0

dtype: int64
```

```
feature_cols = ["gre", "gpa", "rank"]
x = df[feature_cols]
y = df.admit
print(x)
print(y)
```

```

   gre  gpa  rank
0  380.0  3.61  3.0
1  660.0  3.67  3.0
2  800.0  4.00  1.0
3  640.0  3.19  4.0
4  520.0  2.93  4.0
..    ...    ...    ...
395  620.0  4.00  2.0
396  560.0  3.04  3.0
397  460.0  2.63  2.0
398  700.0  3.65  2.0
399  600.0  3.89  3.0

[400 rows x 3 columns]
0      0.0
1      1.0
2      1.0
3      1.0
4      0.0
...
395     0.0
```

```
396 0.0
397 0.0
398 0.0
399 0.0
Name: admit, Length: 400, dtype: float32
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_state=0)
print(x_train.shape,y_train.shape,x_test.shape,y_test.shape)
```

```
(300, 3) (300,) (100, 3) (100,)
```

```
model = LogisticRegression(solver='lbfgs',max_iter=1000)
model.fit(x_train,y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)
```

```
y_pred = model.predict(x_test)
print(y_pred)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 1. 0. 1. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 1. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 1. 0. 0. 1. 0. 0. 1. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 1. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0.]
```

```
conf_mat = metrics.confusion_matrix(y_test,y_pred)
print(conf_mat)
accuracy = metrics.accuracy_score(y_test,y_pred)
print(accuracy)
```

```
[[61  4]
 [27  8]]
0.69
```

```
conf_mat = pd.crosstab(y_test,y_pred,rownames=["Actual"],colnames=["Predicted"])
sns.heatmap(conf_mat,annot=True)
```

<Axes: xlabel='Predicted', ylabel='Actual'>

